

1. Confidence scores for target ID (full probs, no bare labels)

Verdict: **Fully**

Full in the core Kafka classifier contracts (identity_label + confidence + full class_probabilities summing to 1).

2. Threat classifier as capability slot (versioned container, API contract, Helm/K8s)

Verdict: **Partial**

Versioned workers + Docker + Kubernetes manifests exist in the generic Kafka A/B stack (ab_kafka_shadow_slot_enterprise), with a published Pydantic/JSON-style model_output contract — suitable as a pattern for a threat classifier, but not a dedicated “threat CLS” product tree in this workspace (no threat-cls/ directory; only references in tests/audit text).

Helm appears for edge-ai-k3s-final; the Kafka bundle uses Kustomize YAML, not Helm — still satisfies “K8s manifest” if Helm is not mandatory.

3. A/B deployment on Kafka (parallel versions, own topics, comparison)

Verdict: **Fully**

README and code describe two workers, versioned output topics, shared input, classification.comparison, primary alias, etc. (already summarized in ab_kafka_shadow_slot_enterprise/README.md).

4. Calibrated confidence ($ECE \leq 0.05$) + human-review threshold

Verdict: **Partial**

Human review + configurable thresholds + queue refs are implemented in confidence_routing_layer (see tests around human_review_required / review_queue_ref).

Strict calibration so reported confidence matches empirical accuracy with $ECE \leq 0.05$ is not implemented as a fitting/validation loop here. confidence_routing_layer assumes probabilities are already calibrated upstream (see docstring on ClassifierOutputInput).

In aces-confidence-engine, the field named expected_calibration_error in calibration_monitor is not standard binned ECE; it is $|\text{raw_top_score} - \text{calibrated_confidence}|$ on a single path — a different quantity:

5. Explainability (SHAP or equivalent) + visible in COP detail panel

Verdict: **Partial**

Attribution is Integrated Gradients, not SHAP (stated in `explainability_attribution_layer/README.md`); API models expose `class_probabilities` plus `feature_attributions` / rendered explanation endpoints (`PredictionOutput` in `explainability_attribution_layer/app/models.py`).

COP UI / “detail panel” is not built. There is only a minimal JSON schema for COP-related rendering events in the swap package (`model-swap-acceptance-package/schemas/cop-rendering-events.schema.json`), not an end-to-end COP integration.

6. Bias and drift monitoring (input/output drift, accuracy, per-class, alerts, C2 bus)

Verdict: **Partial**

`ai_monitoring_layer` ingests inference + ground truth, computes feature drift, prediction drift, accuracy, per-class summaries, and fairness-style subgroup metrics, exposes Prometheus metrics, Grafana/Alertmanager assets, and alert handling.

A pluggable `internal_alert_bus_urls` fanout exists — usable as a stand-in for a command/alerts bus, but there is no specific C2 system integration in code beyond configurable HTTP URLs.

7. Five-minute rollback (one command, tested on schedule, runbook)

Verdict: **Fully**

Runbook explicitly targets ≤ 5 minutes RTO; rollback API/scripts and `rollback-drill-cronjob.yaml` (scheduled drill) are present under `model-rollback-package`.

8. Edge AI on K3s (quantized, resource limits, same interface as central)

Verdict: **Partial**

Quantized TorchScript, K3s manifests + Helm, CPU/RAM limits, telemetry, and parity-style gating are delivered in `edge-ai-k3s-final`.

The HTTP JSON contract differs from the Kafka `ModelOutput` shape (e.g. `top_k` vs full `class_probabilities` dict), so “same interface contract as central” is conceptually aligned in README terms but not a byte-for-byte identical schema across central Kafka vs edge HTTP.

9. Auto-scaling on Kafka backlog (KEDA / adapter, min/max, cooldown)

Verdict: **Fully**

autoscale_kafka_lag_layer documents KEDA ScaledObject, lag-driven policy, min/max replicas, hysteresis, HPA rate limits, 5-minute downscale stabilization, and optional Prometheus Adapter path.

10. Model swap test (automated swap, downstream safety, schema registry)

Verdict: **Fully**

model-swap-acceptance-package describes exactly: deploy A, replay, swap to B, replay, structural compare, consumer health checks, Schema Registry compatibility (Confluent/Apicurio ccompat), signed reports, CI gate language.

11. Training data provenance (manifest, sources, license, classification, preprocessing, labeling, hash)

Verdict: **Fully**

ai_data_provenance_layer README and design cover lineage, manifests, hashes, classification/license gates, approvals, k8s hardening, etc., as a standalone service/package — not automatically wired into every other layer's release button, but the capability is implemented there.